

Mobile devices resilient to capture

(MacKenzie & Reiter 2003)

Some statistics (@2010)

→ **1 laptop stolen every 12 seconds**

→ **Survey on 329 organizations:**

⇒ 86400 laptop lost; value: 2.1 B\$

→ Data value is greater than PC value!

⇒ 43% stolen at home

→ **FBI data (2001-2005)**

⇒ 160 stolen laptop

⇒ 10+ with critical (classified) data

⇒ 51: unsure whether contained classified data

→ **Famous R&B singer, Ryan Leslie**

⇒ Offered 1M\$ for his stolen laptop in 2010

⇒ New original songs and videos worth A LOT for him

→ (did not get it back)

Capture resilient device

- **A device that CANNOT be used by other than the rightful owner**
- **Assume “core” of the device is a secret key**
 - ⇒ E.g. to permit decryption, digital signatures, etc
 - ⇒ E.g. SIM card
- **Possible security approaches**
 - ⇒ Lock/Unlock key via passwd (e.g. cipher/decipher it)
 - Weak: dictionary attack likely to succeed
 - ⇒ Store secret in tamper-proof HW box
 - Must have it! Must be robust to side channel analysis
 - ⇒ Dynamically download key from network repository
 - Must trust network repository!
- **Other ideas?**

MacKenzie + Reiter, 2003

- **Assumption: device is connected when used**
- **Solution: involve a “capture-protection server” in the network**
 - ⇒ Server confirms that device remains in owner's possession before permitting usage of key
 - ⇒ SW only, no tamper-proof requirements
- **“capture-protection server” does NOT need to be trusted!**
- **Two approaches:**
 - ⇒ Basic one, standard protocol
 - ⇒ Extended one, uses (2,2) secret sharing
 - Even if attacker cracks the device (and the user password), device key can be disabled

Scenario

(MacKenzie & Reiter)



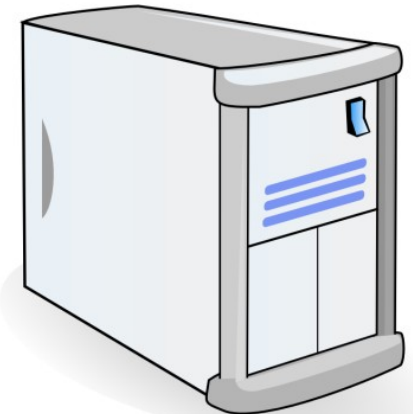
Password
d P

Many possible attack scenarios

- password known
- device stolen
- **server cracked (no trusted server!)**

Good: Solution resilient to any crack

Better: Solution resilient to more than 1 crack!



Secret Key

SKd

Public Key

Giuseppe Bianchi

Secret Key

Public Key

PKs

SKs

Basic solution

→ **Robust to following attacks:**

- ⇒ Server cracked AND password known
 - Attacker cannot sign/decrypt
- ⇒ Device cracked/stolen
 - Attacker can only perform dictionary attack
ONLINE
- ⇒ Device and Server cracked
 - Attacker can only perform dictionary attack
OFFLINE

→ **Basic solution NOT robust to:**

- ⇒ Device AND password cracked
 - Game over... Attacker can do all

Idea: use “tickets”

→ **Assumption:**

⇒ device has access to network

→ **Protection:**

⇒ Encrypt device key so that it can be decrypted only via cooperation with server

→ **Idea:**

⇒ Send encrypted “ticket” to server

→ Contains authentication material for user!!

→ No need to store on server

⇒ Use content of ticket to authenticate user

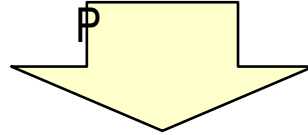
⇒ Use content of ticket to “partially” decrypt device key

→ Final decryption at User – server never sees key!

Protocol: device initialization

Passwd:

P



Password

P

PK_{serv}

Gen random key v

Gen random key a

Hash password

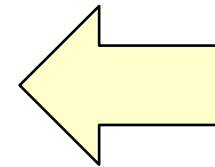
$b = H(P)$

Crypt Secret Key $c = Sk_{device} \oplus PRF(v; P)$

Create ticket

$tk = E_{pk-}$

$server[a, b, c]$



Server
Public
Key

Delete all
red

Device public key PK_{device}

Device Secret key SK_{device}

Protocol: key retrieval

Device

random key v
 random key a
 ticket $\text{tk} = E_{pk-}$

server $[a, b, c]$

$$b = H(P)$$

$$c = \text{Sk}_{\text{device}} \oplus \text{PRF}(v; P)$$

Device public key PK_{device}

User input password
 P

Compute hash
 $\beta = H(P)$

Gen random ρ

$\text{mac}_a[\text{tk}, E_{pk-\text{server}}[\beta, \rho]]$

Server

tk $\rightarrow a, b, c$
 $\rightarrow$ check
 MAC

(auth

device!)

decrypt $\rightarrow \beta, \rho$

Check $\rightarrow b = \beta$

(auth user!)

Decrypt key

$(\rho \oplus c) \oplus \rho \oplus \text{PRF}(v, P) \rightarrow$

$\text{Sk}_{\text{device}}$

$\rho \oplus c$

return

Attacks

→ **Server cracked AND password known**

- ⇒ Key v in device, still secret
 - SK encrypted with $\text{PFR}(v, \text{passwd})$
 - SK cannot be obtained

→ **Device cracked/stolen**

- ⇒ Attacker must send valid passwd hash
- ⇒ can only perform dictionary attack ONLINE
 - Easy to detect!
 - MAC verified (attacker knows a), but passwd fails many times

→ **Device and Server cracked**

- ⇒ Passwd still missing
- ⇒ Dictionary attack OFFLINE against $b = H(\text{Passwd})$

Can we do better?

→ **Limitation**

- ⇒ device stolen and passwd known
 - Attacker can get SK and use it from now on

→ **Solution:**

- ⇒ Must NOT reveal SK to the device itself!
- ⇒ Easy (now 😊): use secret sharing!

→ **Following example:**

- ⇒ RSA signature
- ⇒ Analogous for encryption

Secret sharing (2,2) for RSA

$$n = p \cdot q$$

$d \leftarrow$ random, secret key

$d_1 \leftarrow$ random, share 1

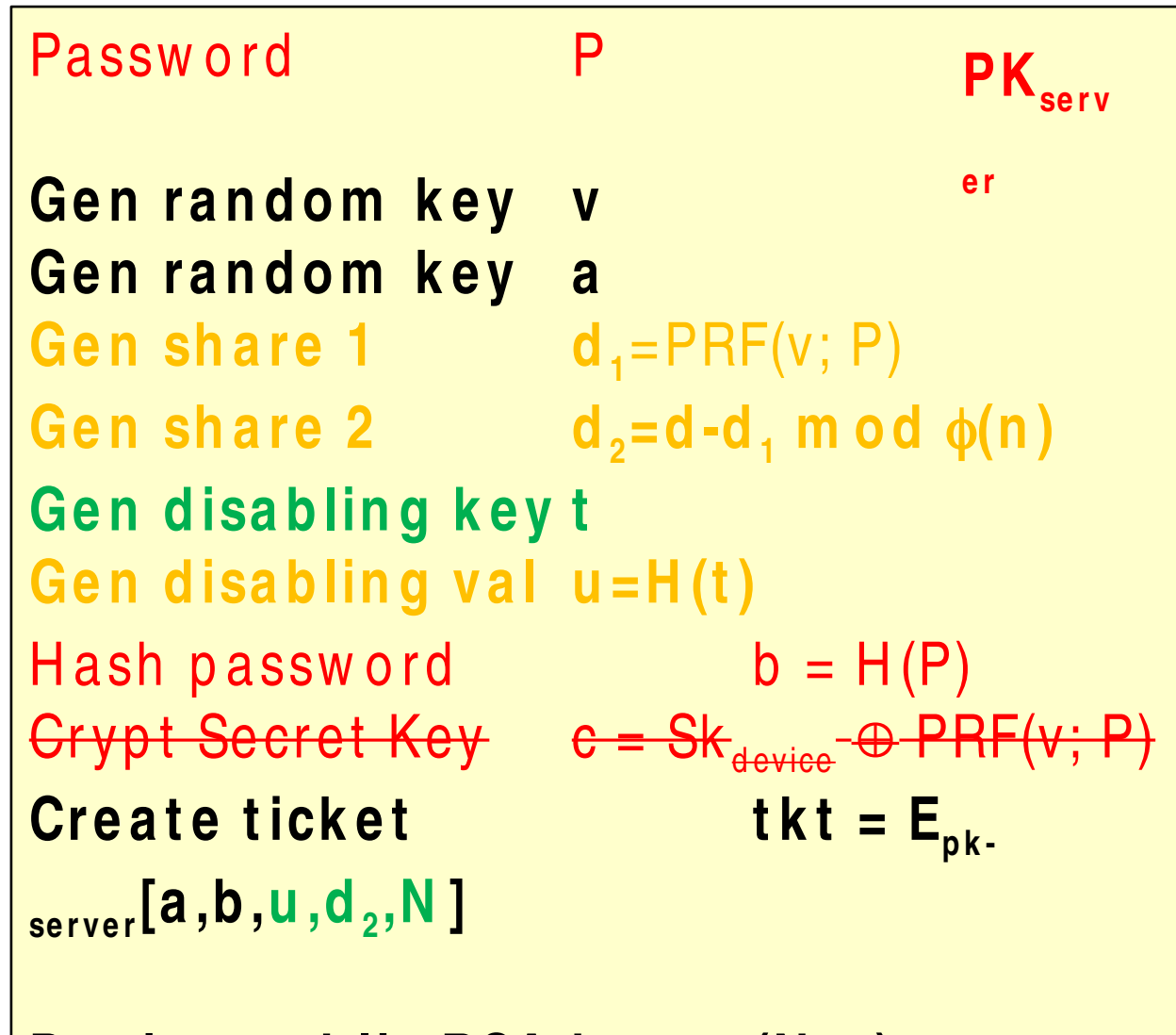
$d_2 = d - d_1 \bmod \phi(n) \leftarrow$ share 2

$$H(m)^{d_1} \cdot H(m)^{d_2} = H(m)^{d_1+d_2} = H(m)^{d_1+d-d_1} = H(m)^d \bmod n$$

Case (2,2): use trivial secret share

NO Shoup's problem to overcome in reconstruction (no Lagrange now !!)

Protocol2: device initialization



Server
Public
Key

Delete all red &
orange

Device public RSA key (N, e)

Device Secret RSA key d [and

Protocol2: key retrieval

Device – while signing

m

random key v

random key a

disabling key t

ticket $\text{tk} = E_{pk-}$

$b = H(P)$

$d_1 = \text{PRF}(v;$

$P)$

User input password

$\text{server}[a, b, u, d_2, N]$

Device public key

(e, N)

Compute hash

$\beta = H(P)$

Gen random

ρ

send

$\text{mac}_a[\text{tk}, E_{pk-\text{server}}[H[m],$

$\beta, \rho]]$

Server

tk

a, b, d_2, u, N

a

→ check

MAC

(auth

device!)

decrypt → $H[m], \beta,$

return

ρ

Check → $b = \beta$

(auth user!)

Decrypt key

$(\rho \oplus H[m]^{d_2}) \oplus$

ρ

Complete

signature

$d_1 = \text{PRF}(v, P)$

$H[m]^{d_2} H[m]^{d_1}$

$\rho \oplus H[m]^{d_2}$

Signature key d NEVER retrieved in clear!

If attacker gets passwd AND device,

cannot get key d

Giuseppe Bianchi

Protocol2: key disabling

→ Suffices to keep backup of

⇒ t

⇒ tk

→ If device stolen, send them to server

⇒ Server keeps blacklist

